Techniques for Handling Imbalanced Data:

Class Weighting Method:

```
classifier = RandomForestClassifier(
   class_weight='balanced', // Just add class weight is balanced
   random_state=42
)
```

you pass class_weight=**'balanced'**, sklearn automatically calculates weights

weight for class i = total samples / (number of classes × samples in class i)

Here, Not Purchased = 79

Purchased = 41

Total samples = 79 + 41 = **120**

Number of classes = **2 (Purchased and Not Purchased)**

Class 1:

weight for class = 120/ (2 * 79)

= 120 / 158 = 0.759

Class 2:

weight for class = 120/ (2 * 41)

= 120 / 82 = 1.463

Class	**Samples**	**Weight**	**Meaning**
0 - Not Purchased	79	0.759	Lower weight (majority)
1 - Purchased	41	1.463	Higher weight (minority)

The smaller class always receives a higher weight. Since the 'Purchased' class has a count of 41, which is lower than the other class, it is assigned a higher weight.

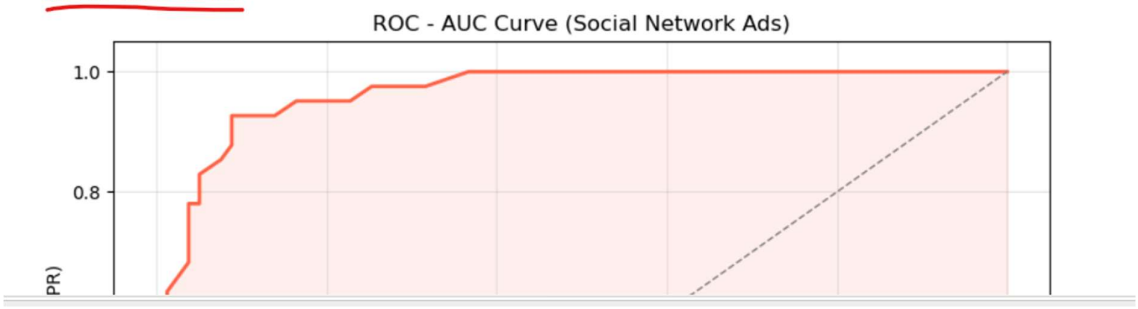
Minority class gets more attention; majority class gets less → learning becomes balanced.

Overall balance -> Achieved without adding/removing data

With Class Weighting Method

```
plt.fill_between(fpr, tpr, alpha=0.1, color='tomato')
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('ROC - AUC Curve (Social Network Ads)')
plt.legend(loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('roc_auc_curve.png', dpi=150, bbox_inches='tight')
plt.show()
```

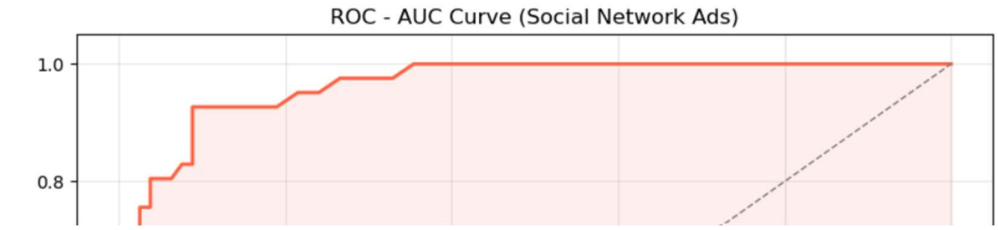
```
ROC-AUC Score: 0.9648
```



Without **Class Weighting Method**

```
# Step 4 – Plot the ROC Curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='tomato', lw=2,
         label=f'ROC Curve (AUC = {auc_score:.4f})')
plt.plot([0, 1], [0, 1], color='gray', lw=1,
         linestyle='--', label='Random Guess (AUC = 0.50)')
plt.fill_between(fpr, tpr, alpha=0.1, color='tomato')
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('ROC - AUC Curve (Social Network Ads)')
plt.legend(loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('roc_auc_curve.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
ROC-AUC Score: 0.9606
```



Oversampling:

Here First, Not purchase 178 Samples, Purchase is 102

After Oversampling, Not purchase 178 Samples, Purchase is 178

```
Name: count, dtype: int64
```

[2]:
```
from imblearn.over_sampling import RandomOverSampler
from sklearn.ensemble import RandomForestClassifier

ros = RandomOverSampler(random_state=42)
X_train_ros, Y_train_ros = ros.fit_resample(X_train, Y_train)

print("Before:", Y_train.value_counts().to_dict())
print("After :", pd.Series(Y_train_ros).value_counts().to_dict())

classifier = RandomForestClassifier(random_state=42)
classifier.fit(X_train_ros, Y_train_ros)

Y_pred = classifier.predict(X_test)
```

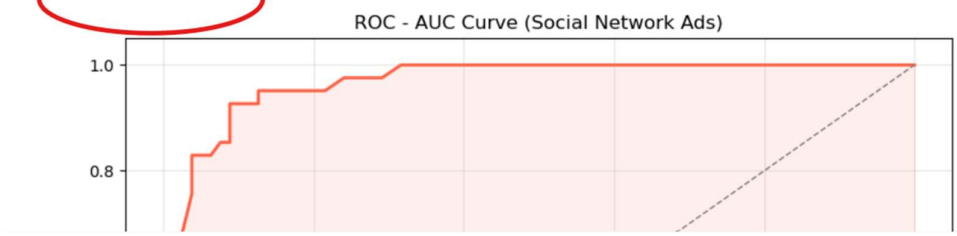
```
Before: {0: 178, 1: 102}
After : {0: 178, 1: 178}
```

[3]:
```
from sklearn.metrics import classification_report
print(classification_report(Y_test, Y_pred))
```

```
              precision    recall  f1-score   support

           0       0.96      0.91      0.94        79
           1       0.84      0.93      0.88        41

    accuracy                           0.92       120
   macro avg       0.90      0.92      0.91       120
weighted avg       0.92      0.92      0.92       120
```

[4]:
```
import matplotlib.pyplot as plt
from sklearn.metrics import roc_auc_score, roc_curve

# Step 1 – Get probability scores for class 1 (Purchased)
Y_pred_proba = classifier.predict_proba(X_test)[:, 1]

# Step 2 – Calculate AUC score
# Step 3 – Get TPR and FPR values for plotting
fpr, tpr, thresholds = roc_curve(Y_test, Y_pred_proba)

# Step 4 – Plot the ROC Curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='tomato', lw=2,
         label=f'ROC Curve (AUC = {auc_score:.4f})')
plt.plot([0, 1], [0, 1], color='gray', lw=1,
         linestyle='--', label='Random Guess (AUC = 0.50)')
plt.fill_between(fpr, tpr, alpha=0.1, color='tomato')
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('ROC - AUC Curve (Social Network Ads)')
plt.legend(loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('roc_auc_curve.png', dpi=150, bbox_inches='tight')
plt.show()
```

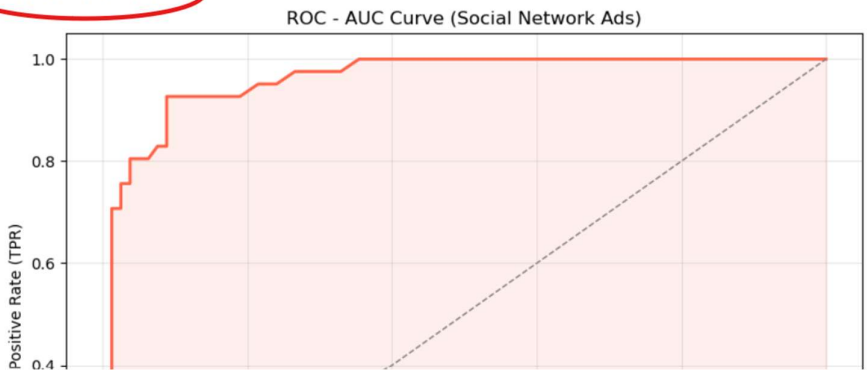
```
ROC-AUC Score: 0.9646
```



Without Oversampling:

```
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='tomato', lw=2,
         label=f'ROC Curve (AUC = {auc_score:.4f})')
plt.plot([0, 1], [0, 1], color='gray', lw=1,
         linestyle='--', label='Random Guess (AUC = 0.50)')
plt.fill_between(fpr, tpr, alpha=0.1, color='tomato')
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('ROC - AUC Curve (Social Network Ads)')
plt.legend(loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('roc_auc_curve.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
ROC-AUC Score: 0.9606
```



SMOTE:

Here First, Not purchase 178 Samples, Purchase is 102

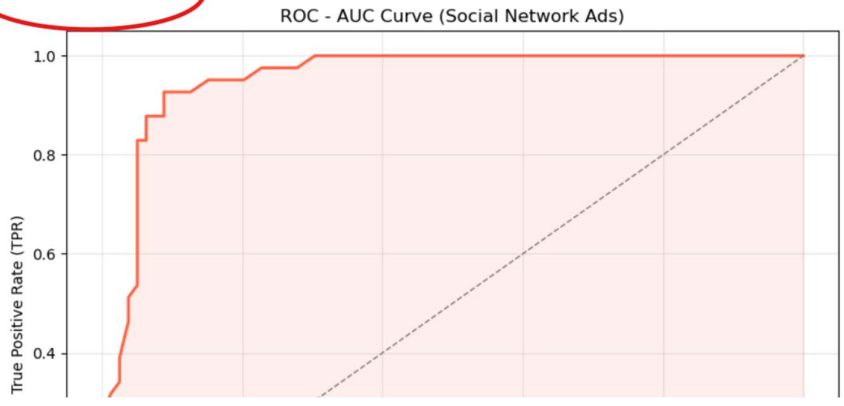
After Oversampling, Not purchase 178 Samples, Purchase is 178

[6]:
```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
X_train_sm, Y_train_sm = smote.fit_resample(X_train, Y_train)

print("Before SMOTE:", Y_train.value_counts().to_dict())
print("After Y_train  SMOTE:", pd.Series(Y_train_sm).value_counts())

# Train model
from sklearn.ensemble import RandomForestClassifier
classifier = RandomForestClassifier(random_state=42)
classifier.fit(X_train_sm, Y_train_sm)

Y_pred = classifier.predict(X_test)

# Evaluation
from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_auc_score)

print("\nConfusion Matrix:")
print(confusion_matrix(Y_test, Y_pred))

print("\nClassification Report:")
print(classification_report(Y_test, Y_pred))
```

```
Before SMOTE: {0: 178, 1: 102}
After Y_train  SMOTE: Purchased
0    178
1    178
Name: count, dtype: int64

Confusion Matrix:
[[72  7]
```

```
        linestyle='--', label='Random Guess (AUC = 0.50)')
plt.fill_between(fpr, tpr, alpha=0.1, color='tomato')
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('ROC - AUC Curve (Social Network Ads)')
plt.legend(loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('roc_auc_curve.png', dpi=150, bbox_inches='tight')
plt.show()
```

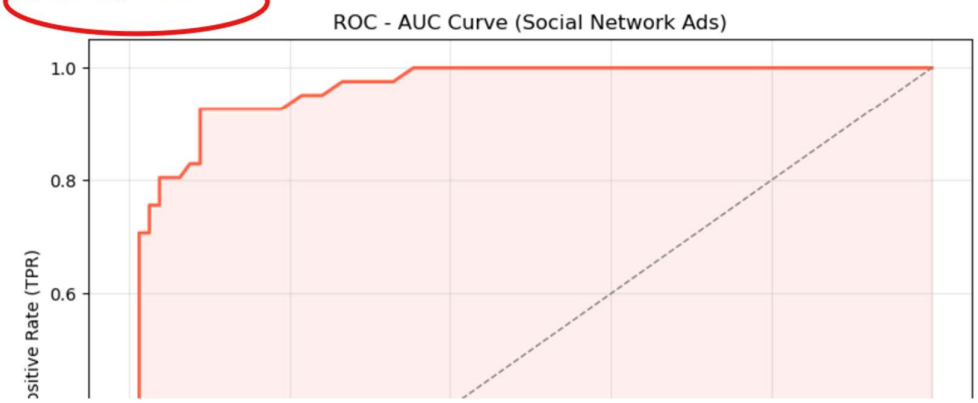
```
ROC-AUC Score: 0.9548
```



Without SMOTE:

```python
# Step 3 – Get FPR and TPR values for plotting
fpr, tpr, thresholds = roc_curve(Y_test, Y_pred_proba)

# Step 4 – Plot the ROC Curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='tomato', lw=2,
         label=f'ROC Curve (AUC = {auc_score:.4f})')
plt.plot([0, 1], [0, 1], color='gray', lw=1,
         linestyle='--', label='Random Guess (AUC = 0.50)')
plt.fill_between(fpr, tpr, alpha=0.1, color='tomato')
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('ROC - AUC Curve (Social Network Ads)')
plt.legend(loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('roc_auc_curve.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
ROC-AUC Score: 0.9606
```



Under Sampling:

Here First, Not purchase 178 Samples, Purchase is 102

After Oversampling, Not purchase 102 Samples, Purchase is 102

```
Name: count, dtype: int64
```

[2]:
```
from sklearn.ensemble import RandomForestClassifier
from imblearn.under_sampling import RandomUnderSampler

rus = RandomUnderSampler(random_state=42)
X_train_rus, Y_train_rus = rus.fit_resample(X_train, Y_train)

print("Before: ", Y_train.value_counts().to_dict())
print("After :", pd.Series(Y_train_rus).value_counts().to_dict())

classifier = RandomForestClassifier(random_state=42)
classifier.fit(X_train_rus, Y_train_rus)

Y_pred = classifier.predict(X_test)
```

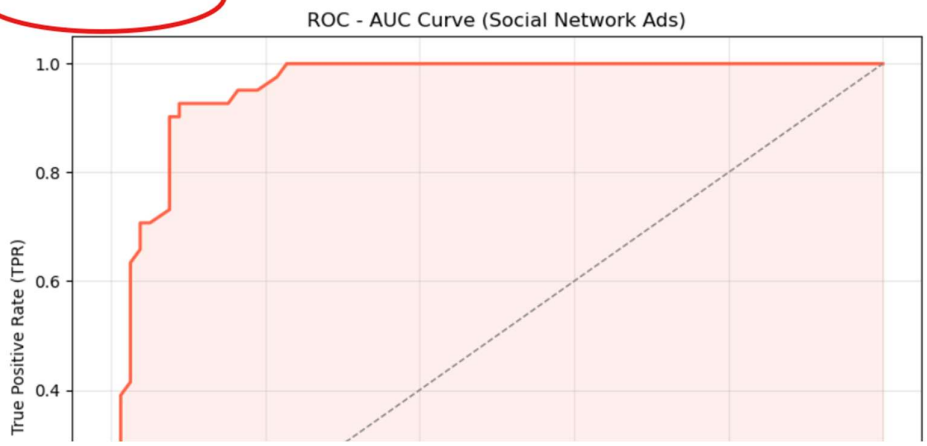
```
Before: {0: 178, 1: 102}
After : {0: 102, 1: 102}
```

[3]:
```
from sklearn.metrics import classification_report
print(classification_report(Y_test, Y_pred))
```

```
              precision    recall  f1-score   support

           0       0.96      0.91      0.94        79
```

```
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('ROC - AUC Curve (Social Network Ads)')
plt.legend(loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('roc_auc_curve.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
ROC-AUC Score: 0.9571
```



Without UnderSampling:

```python
# Step 4 – Plot the ROC Curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='tomato', lw=2,
         label=f'ROC Curve (AUC = {auc_score:.4f})')
plt.plot([0, 1], [0, 1], color='gray', lw=1,
         linestyle='--', label='Random Guess (AUC = 0.50)')
plt.fill_between(fpr, tpr, alpha=0.1, color='tomato')
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('ROC - AUC Curve (Social Network Ads)')
plt.legend(loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('roc_auc_curve.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
ROC-AUC Score: 0.9606
```